

Machine Learning in Practice I

Lecture 6

Antal Jakovac, 2025

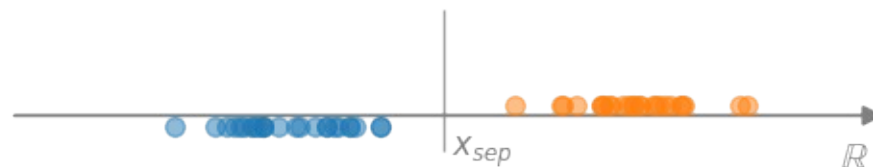
Data modelling

- task: find out the selective/descriptive features
- depends both on the dataset and the context
- selective: constant on a class, descriptive: changes within a class
class depends on the context
- finding the selective coordinates is the main challenge
- we start with simple 1D classification problems, and generalize to several dimensions

One dimensional dataset classification

One dimensional dataset

- Simplest basic case: characterization of one-dimensional data
 - data $\rightarrow x \in \mathbb{R}$ (numbers for analytic characterization)
- Separation of two 1D sets
 - assume that in given sections belong (approximately) to a single class
 - basic case: one section $\rightarrow A$, other section $\rightarrow B$
 - if the two sets are separated (hard margin) $\left\{ \begin{array}{l} x \in A \Leftrightarrow x < x_{sep} \\ x \in B \Leftrightarrow x > x_{sep} \end{array} \right\}$
 - overlapping case? $\rightarrow$ no perfect solution



One dimensional dataset

- overlapping regions → no perfect solution (heuristics)

→ choose x_{sep} somewhere in the overlapping region (eg. midpoint)

→ define an error function in the overlapping region, e.g.

→ $p=1$ (“hinge loss”) → median, $p=2$ → mean

$$L_p(x_{sep}) = \sum_{x \in \text{overlap}} |x - x_{sep}|^p$$

→ we can weight the points



$$0 = \frac{d}{dx_{sep}} \sum_{i=1}^N (x_i - x_{sep})^2 = 2 \sum_{i=1}^N (x_{sep} - x_i) \Rightarrow x_{sep} = \frac{1}{N} \sum_{i=1}^N x_i$$

$$\sum_{i=1}^N |x_i - x_{sep}| = \sum_{x_i > x_{sep}} (x_i - x_{sep}) + \sum_{x_i < x_{sep}} (x_{sep} - x_i) = (N_{<} - N_{>}) x_{sep} + \sum_{x_i > x_{sep}} x_i - \sum_{x_i < x_{sep}} x_i \Rightarrow N_{<} = N_{>}$$

- Another approach: treat the problem as a regression

→ define a function $f: \mathbb{R} \rightarrow \mathbb{R}$, that
$$\begin{cases} f(x) < 0 \Leftrightarrow x \in A \\ f(x) > 0 \Leftrightarrow x \in B \end{cases}$$

→ a classification can always be translated to regression

→ simplest: linear relation $f(x) = ax + b$

→ probabilistic interpretation: $p(x) = \frac{e^{f(x)}}{1 + e^{f(x)}} \in [0, 1], \quad p(0) = 0.5$

logistic map, probability of belonging to class B

→ loss function:
$$L(a, b) = \sum_x (p(x) - I_{x \in B})^2, \quad I_{x \in B} = 1 \text{ if } x \in B, \text{ else } 0$$

One dimensional dataset

- Another approach: tree

- define a function $f: \mathbb{R} \rightarrow \mathbb{R}$

- a classification can always be done

- simplest: linear relation

- probabilistic interpretation

logistic map, probability of belonging to class B

- loss function:
$$L(a, b) = \sum_x (p(x) - I_{x \in B})^2, \quad I_{x \in B} = 1 \text{ if } x \in B, \text{ else } 0$$

```
# Python function minimization
```

```
import numpy, matplotlib
```

```
def P(p, x):
```

```
    a, x0 = p
```

```
    return np.exp(a*(x-x0))/(1+np.exp(a*(x-x0)))
```

```
def loss(p, A, B):
```

```
    loss = np.sum([P(p, a)**2 for a in A])
```

```
    loss += np.sum([(P(p, b)-1)**2 for b in B])
```

```
    return loss
```

```
p0 = [1, 1] # initial guess
```

```
params = (A, B) # values of a, b
```

```
res = minimize(loss, p0, args=params)
```

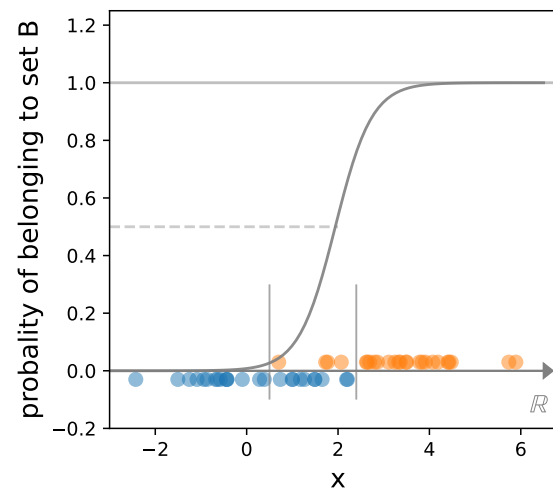
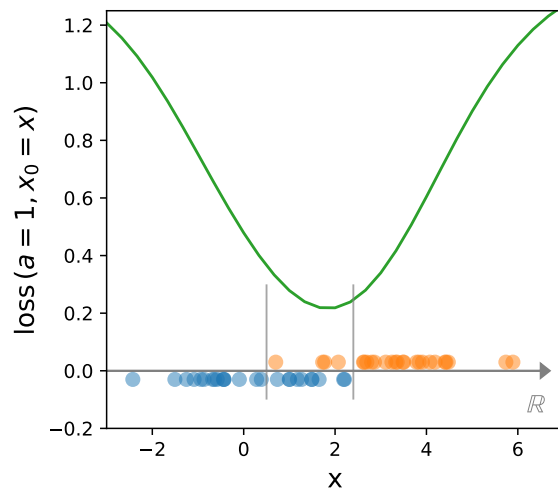
```
a, x0 = res.x
```


One dimensional dataset

- Another approach: treat the problem as a regression

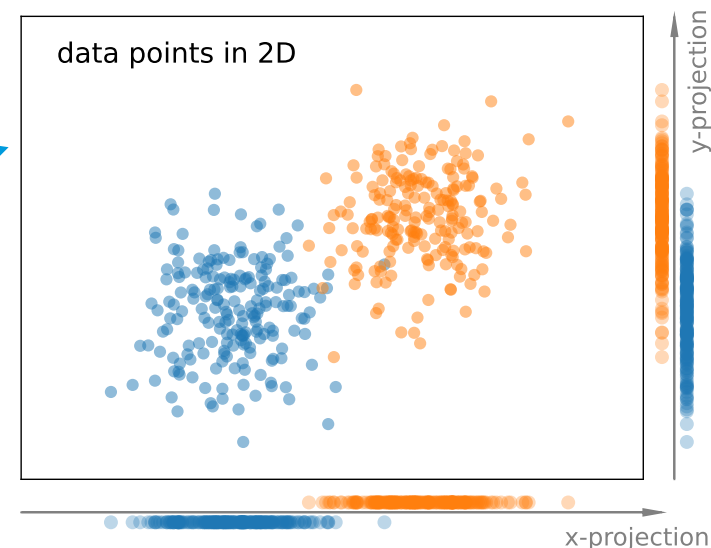
- ➔ logistic map, probability of belonging to class B

$$p(x) = \frac{e^{ax+b}}{1+e^{ax+b}}$$

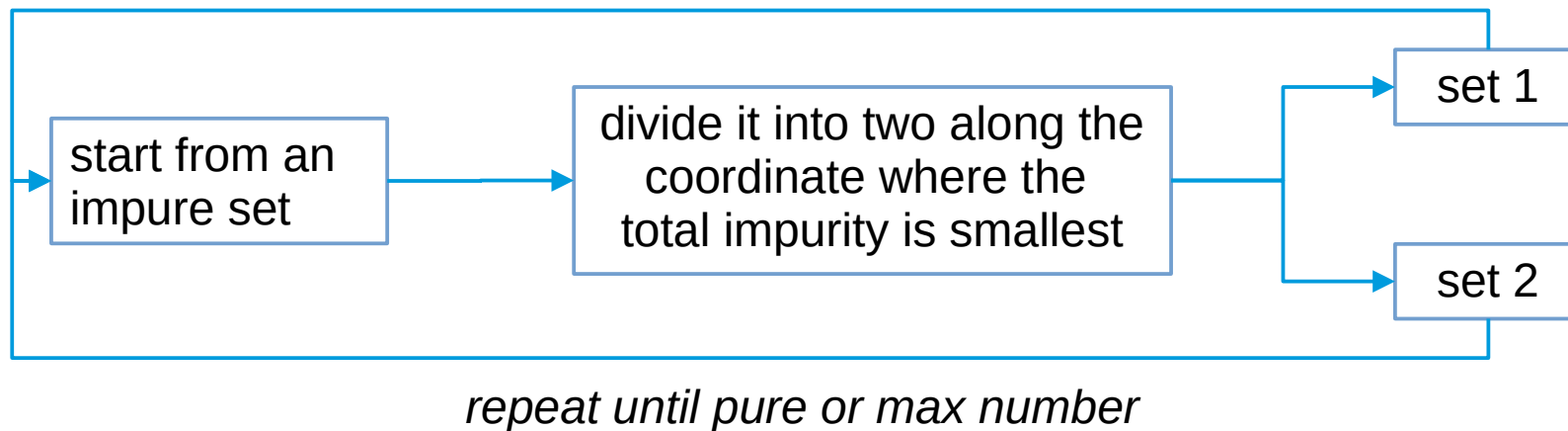


Decision trees

- The **generalization to multiple dimensions** is not simple
 - in 1D the coordinates are unique
 - in multidimensional case we can optimize the coordination → features
 - simplest case: use original coordinates
 - usually the original coordinates are not equally useful
(c.f. *image* → *x coord separates more*)
 - choose that coordinate that leads to the smallest total impurity!



- Decision tree (DT)



- always converges, but may result in fragmented divisions

- Decision tree (DT) example

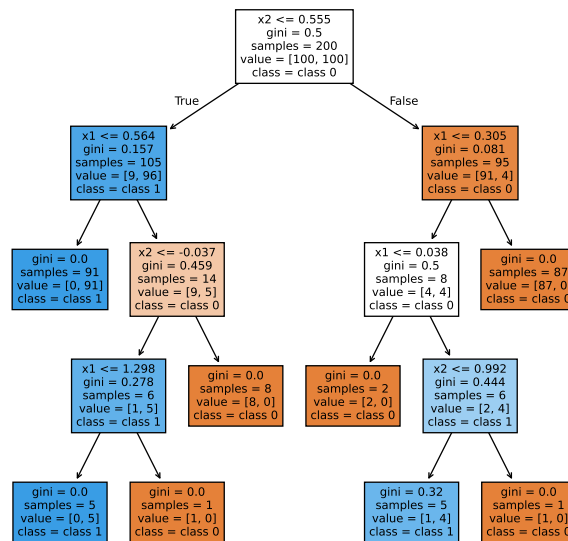
 - two 2D sets

- pros:

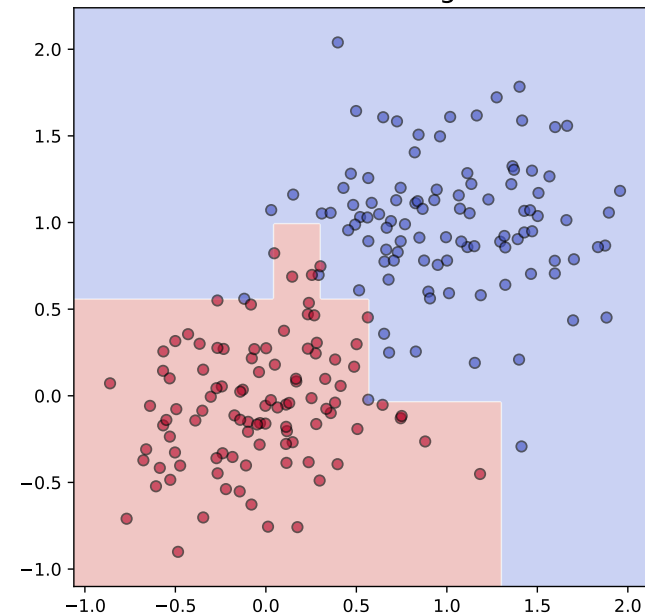
 - simple
 - interpretable

- cons:

 - not robust enough
($\rightarrow$ *irregular shape*)



Decision Tree regions



Tree based methods

- Decision tree (DT) example

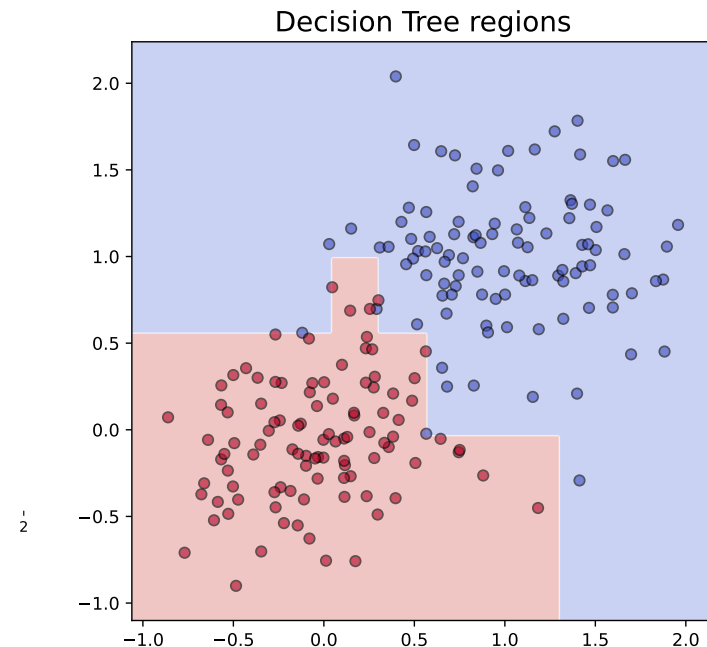
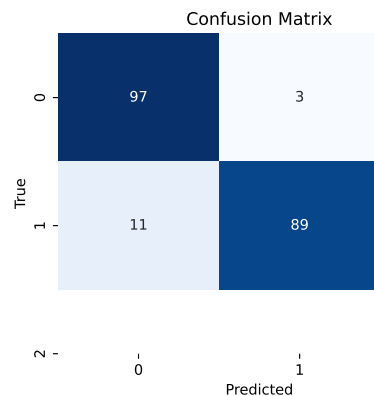
 - two 2D sets

- pros:

 - simple
 - interpretable

- cons:

 - not robust enough
(→ *irregular shape*)
use fixed depth!
 - accuracy: *train: 100%, test: 92.5%*





- Decision tree (DT) technical solution in Python → [github](#)

```
# Decision Tree example  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import confusion_matrix  
  
# 1. get or generate data and labels  
  
# 2. Fit a decision tree  
clf = DecisionTreeClassifier()  
clf.fit(data, labels)  
  
# 3. use for prediction  
pred_labels = clf.predict(other_data)  
  
#4. confusion matrix  
cm = confusion_matrix(labels, pred_labels)
```

Ensemble methods

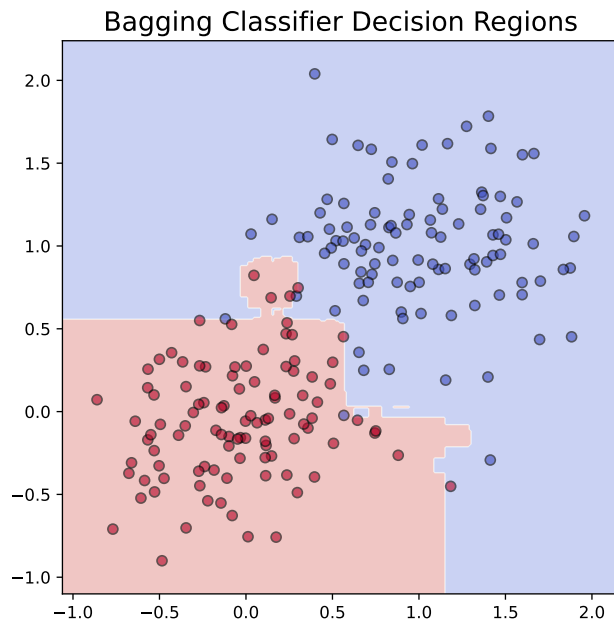


- Decision tree → too specific for a sample
- Possible solution: use ensemble of methods
- ways of improvement
 - bagging (Bootstrap AGGregatING) → bagging, random forest
 - boosting → adaptive boosting, gradient boosting
 - other methods → extra trees, isolated forests, oblique trees



- Idea:
 - train several models in slightly different datasets
 - each dataset contains the same number of data from the database, but some of them are missing, some appear multiple times
 - combine the results → regression: average, classification: majority vote
- Benefits:
 - lowers variance
 - increases robustness

- Code: [github](#)



Bagging example

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix
```

bagging based on decision tree

```
base_tree = DecisionTreeClassifier()
clf = BaggingClassifier(
    estimator=base_tree,
    n_estimators=50,           # number of trees
    max_samples=0.8,          # each tree sees 80% of data
    bootstrap=True,           # sampling with replacement
    random_state=42
)
clf.fit(X,y)
```

3. use for prediction

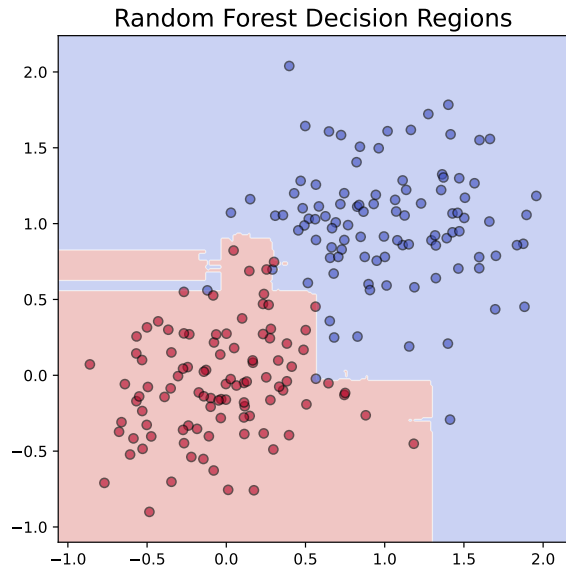
```
pred_labels = clf.predict(other_data)
```

#4. confusion matrix

```
cm = confusion_matrix(labels, pred_labels)
```

- **Idea:** in addition vary the used parameters, too
 - train several models in slightly different datasets
 - each model uses selected data and selected parameters
 - combine the results → regression: average, classification: majority vote
- **Benefits:**
 - more robust than simple bagging
 - can be used to rank parameters by their significance, i.e. which contributed more to the final decision

- 2D case
- accuracy: 94%



```
# Random Forest example
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
# bagging based on decision tree

# Train Random Forest
clf = RandomForestClassifier(
    n_estimators=100, # number of trees
    max_depth=None, # let trees grow fully
    random_state=42 # random generator seed
)
clf.fit(X, y)

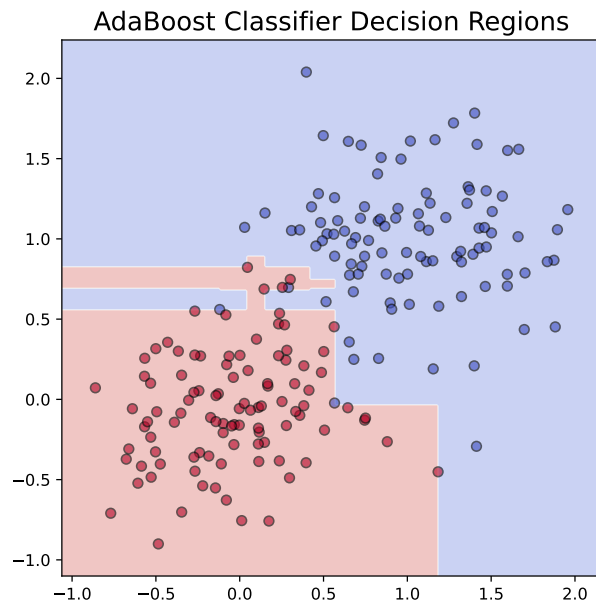
# 3. use for prediction
pred_labels = clf.predict(other_data)

#4. confusion matrix
cm = confusion_matrix(labels, pred_labels)
```



- Idea:
 - calculate a series of decision trees
 - each focuses on the points where the previous one fails
 - combine weighted sum of all of these models
- Adaptations: AdaBoost, Gradient Boosting (GBM), XGBoost, ...

- 2D case
- accuracy: 94%



AdaBoost example

```
from sklearn.metrics import confusion_matrix  
from sklearn.ensemble import AdaBoostClassifier
```

boosting based on decision tree

Train AdaBoost model

```
boost = AdaBoostClassifier(  
    estimator=DecisionTreeClassifier(max_depth=2),  
    n_estimators=50,  
    learning_rate=1.0,  
    random_state=42)  
boost.fit(X, y)
```

3. use for prediction

```
pred_labels = boost.predict(other_data)
```

#4. confusion matrix

```
cm = confusion_matrix(labels, pred_labels)
```

END OF LECTURE 6